

PROGETTO FINALE

Gli studenti possono **scegliere uno dei due progetti disponibili**. Entrambi i progetti hanno **lo stesso peso nella valutazione**.

Progetto 1: SISTEMA DI TRANSAZIONI BANCARIE DISTRIBUITE CON MUTUA ESCLUSIONE A TOKEN RING

1. Obiettivo del Progetto

L'obiettivo di questo progetto è **progettare, implementare e dimostrare un sistema distribuito** che garantisca la **mutua esclusione** utilizzando l'**algoritmo Token Ring**.

Il sistema simula un **ambiente bancario distribuito** in cui più nodi indipendenti (ATM) devono accedere e modificare in modo sicuro un **saldo bancario condiviso**.

Il progetto valuta sia:

- **La correttezza dell'algoritmo distribuito**
- **La qualità dell'implementazione, della documentazione e della spiegazione**

2. Vincoli Generali

- Il progetto **può essere implementato in qualsiasi linguaggio di programmazione**
- Il sistema è composto da **esattamente 4 nodi**
- Tutti i nodi devono essere **simulati su localhost**
- Ogni nodo deve essere eseguito in una **console / terminale separata**
- Tutto il coordinamento deve avvenire tramite **scambio di messaggi**
- La mutua esclusione deve essere implementata **esclusivamente tramite l'algoritmo Token Ring**

3. Panoramica del Sistema

- Ogni nodo rappresenta un **ATM**
- Tutti gli ATM operano su **un unico conto bancario condiviso**
- Le transazioni includono:
 - Deposito
 - Prelievo
- Ogni transazione costituisce una **sezione critica**
- Solo il nodo che possiede il token può eseguire una transazione

4. Topologia Logica

I nodi formano un **anello logico**:

ATM1 → ATM2 → ATM3 → ATM4 → ATM1

- L'anello è **logico**, non fisico
- Tutta la comunicazione avviene su localhost
- Ogni nodo conosce solo il proprio **successore**

5. Risorsa Condivisa e Transazioni

5.1 Risorsa Condivisa

- La risorsa condivisa è il **saldo del conto bancario**
- Non esiste memoria condivisa: **I 4 nodi NON possono leggere o scrivere direttamente le stesse variabili in memoria**. In altre parole:
 - Ogni nodo (ATM) è un **programma separato**
 - Ogni programma ha **la propria memoria privata**
 - Nessun nodo può fare cose come:

`balance = balance - 100; // leggendo una variabile globale comune`

- Gli aggiornamenti devono essere serializzati tramite mutua esclusione

5.2 Transazioni

Una transazione consiste nelle seguenti operazioni:

1. Lettura del saldo corrente
2. Validazione dell'operazione
3. Aggiornamento del saldo
4. Scrittura del nuovo valore
5. Registrazione (log) dell'operazione

Ogni transazione deve essere eseguita in modo **atomico**.

6. Modello di Esecuzione Token Ring

1. Nel sistema esiste **esattamente un token**
2. Il token circola continuamente lungo l'anello
3. Il possesso del token concede il permesso di:
 - Entrare nella sezione critica
 - Eseguire una transazione
4. Se un nodo non necessita della sezione critica, inoltra immediatamente il token

7. Esempio Dettagliato di Transazioni

Stato Iniziale

- Saldo iniziale: **1000**
- Nodo che possiede inizialmente il token: **ATM1**

Richieste di Transazione

Nodo.....Transazione
ATM2.....Prelievo 200
ATM3.....Deposito 100
ATM4.....Prelievo 500

Sequenza di Esecuzione

1. ATM1

- Possiede il token
- Nessuna transazione
- Inoltra il token

Saldo: 1000

2. ATM2

- Riceve il token
- Esegue un prelievo di 200

Saldo: 800

3. ATM3

- Riceve il token
- Esegue un deposito di 100

Saldo: 900

4. ATM4

- Riceve il token
- Esegue un prelievo di 500

Saldo: 400

Questa sequenza dimostra:

- Mutua esclusione

- Aggiornamento corretto del saldo
- Ordinamento basato sul token

9. Requisiti di Logging

Ogni nodo deve registrare chiaramente:

- Ricezione del token
- Inoltro del token
- Inizio della transazione
- Fine della transazione
- Saldo aggiornato

I log devono mostrare chiaramente:

- Il movimento del token
- L'assenza di transazioni concorrenti
- Il corretto comportamento del sistema

Consegna Finale

Documento PDF Unico

Gli studenti devono consegnare **un unico file PDF** che includa **tutti** i seguenti elementi:

1. Istruzioni di Esecuzione (Obbligatorie)

Il PDF deve includere:

- Linguaggio di programmazione utilizzato
- Librerie o dipendenze richieste
- Modalità di compilazione (se applicabile)
- Comandi esatti per avviare **ciascuno dei 4 nodi**
- Spiegazione di come i nodi sono associati ai terminali

Le istruzioni devono permettere l'esecuzione del progetto **senza ulteriori chiarimenti**.

2. Link al Repository GitHub Personale (Obbligatorio)

Il PDF deve includere un **link a un repository GitHub personale** contenente:

- Il codice sorgente completo del progetto
- **Commenti sufficienti**, in particolare per:
 - Gestione del token
 - Scambio di messaggi
 - Logica della sezione critica
- Un **video dimostrativo** che mostri chiaramente:
 - Tutti e 4 i nodi in esecuzione su terminali separati
 - La circolazione del token
 - L'esecuzione delle transazioni

Progetto 2: Elezione del Coordinatore con Bully Algorithm

1. Scopo del progetto

Lo scopo del progetto è implementare un sistema distribuito che realizzi **l'Algoritmo del Bullo (Bully Algorithm)** per l'elezione di un coordinatore, mostrando esplicitamente due fasi fondamentali:

1. **Prima elezione**, in cui il nodo con ID massimo (**P5**) viene eletto coordinatore dall'algoritmo.
2. **Seconda elezione**, innescata dal **guasto di P5**, in cui il nuovo coordinatore deve essere **P4**, ovvero il nodo con ID più alto ancora attivo.

2. Configurazione del sistema

Nodi

Il sistema è composto da **5 nodi indipendenti**:

Nodo.....ID

P1.....1

P2.....2

P3.....3

P4.....4

P5.....5

Ogni nodo:

- è un **processo separato**
- è eseguito in un **terminale distinto**
- comunica **solo tramite messaggi** su **localhost**

3. Ambiente di esecuzione

- Tutti i nodi girano su **localhost**
- Comunicazione tramite **socket di rete** (TCP o UDP)
- Nessuna memoria condivisa
- Nessun coordinatore predefinito

Fase 1 – Elezione iniziale del coordinatore

Stato iniziale

- Nessun nodo conosce il coordinatore
- Tutti i nodi sono attivi
- Uno qualunque dei nodi (ad esempio P1) avvia l'elezione

Comportamento richiesto

1. Il nodo che avvia l'elezione invia **ELECTION** a tutti i nodi con ID più alto.

2. Ogni nodo con ID più alto:
 - risponde con **ANSWER**
 - avvia una propria elezione (se non già in corso)
3. Il processo continua finché:
 - il nodo con ID massimo (**P5**) non riceve ANSWER da nodi superiori

Risultato della fase 1

- **P5 si proclama coordinatore**
- P5 invia **COORDINATOR** a tutti i nodi con ID inferiore
- Tutti i nodi registrano P5 come coordinatore
- Il sistema entra in stato stabile

- **P5 deve diventare coordinatore esclusivamente tramite l'algoritmo**, non per assegnazione iniziale.

Fase 2 – Guasto del coordinatore P5

Evento di guasto

- Il processo **P5 viene terminato manualmente** (es. Ctrl+C)
- P5 smette di rispondere ai messaggi

Rilevazione del guasto

- Gli altri nodi tentano di comunicare con P5
- Dopo un **timeout T**, P5 è considerato fallito

Fase 3 – Nuova elezione dopo il guasto

Avvio dell'elezione

- Un nodo (es. P2 o P3) rileva il timeout
- Invia **ELECTION** ai nodi con ID più alto ancora noti (P4 e P5)
- P5 non risponde
- P4 risponde con **ANSWER** e avvia la propria elezione

Meccanismo del “bullo”

- P4 invia **ELECTION** a P5
- Nessuna risposta da P5
- P4 conclude di essere il nodo con ID più alto attivo

Risultato finale

- **P4 si proclama coordinatore**
- P4 invia **COORDINATOR** a P1, P2 e P3
- Tutti i nodi accettano P4 come coordinatore
- Il sistema torna in stato stabile

Messaggi obbligatori

Devono essere implementati **solo** i seguenti messaggi:

- ELECTION
- ANSWER
- COORDINATOR

Timeout richiesti

- **T**: tempo massimo di attesa per risposte ANSWER
- **T'**: tempo massimo di attesa per il messaggio COORDINATOR

I timeout devono essere **realmente implementati**, non simulati.

Requisiti di correttezza

L'implementazione deve dimostrare che:

1. Alla fine della **prima elezione**, il coordinatore è **P5**
2. Dopo il guasto di P5:
 - l'elezione termina
 - il coordinatore è **P4**

3. Non esistono mai due coordinatori contemporaneamente
4. L'algoritmo termina sempre

Output richiesto

Ogni nodo deve stampare su terminale:

- invio e ricezione dei messaggi
- timeout scaduti
- avvio delle elezioni
- proclamazione del coordinatore

Esempio atteso

[P1] Avvio elezione

[P5] Nessun nodo con ID superiore

[P5] Mi dichiaro COORDINATORE

...

*[P5] *** processo terminato ****

[P3] Timeout: coordinatore non risponde

[P4] Nessuna risposta da P5

[P4] Mi dichiaro COORDINATORE

Consegna Finale del Progetto

Formato della Consegna

Gli studenti devono consegnare **un unico documento PDF** contenente **tutti gli elementi descritti** **di** **seguito.**

Non sono accettate consegne parziali o materiali separati.

1. Istruzioni di Esecuzione

Il documento PDF deve includere una sezione chiaramente identificabile intitolata **“Istruzioni di Esecuzione”**, che permetta al docente di **eseguire il progetto senza alcun chiarimento aggiuntivo**.

Questa sezione deve specificare in modo esplicito:

1.1 Linguaggio e Ambiente

- Linguaggio di programmazione utilizzato
- Versione del linguaggio (se rilevante)
- Sistema operativo di riferimento (se necessario)

1.2 Librerie e Dipendenze

- Elenco completo delle librerie o dipendenze richieste
- Indicazione se le librerie sono standard o esterne
- Comandi per l’installazione delle dipendenze (se applicabile)

1.3 Compilazione (se applicabile)

- Comandi esatti per compilare il progetto
- Eventuali file di configurazione necessari

1.4 Avvio dei Nodi

Il PDF deve indicare **esattamente**:

- I comandi per avviare **ciascuno dei 5 nodi (P1, P2, P3, P4, P5)**
- Le porte o gli indirizzi utilizzati su localhost
- L’ordine consigliato di avvio dei nodi (se rilevante)

Esempio (indicativo):

Terminale 1: avvio nodo P1

Terminale 2: avvio nodo P2

Terminale 3: avvio nodo P3

Terminale 4: avvio nodo P4

Terminale 5: avvio nodo P5

1.5 Associazione Nodo–Terminale

Deve essere chiaramente spiegato:

- come ciascun nodo è associato a un terminale
- come riconoscere dai log quale nodo è in esecuzione in ciascun terminale

2. Link al Repository GitHub Personale

Il PDF deve contenere **un link funzionante** a un **repository GitHub personale** dello studente.

Il repository deve includere **obbligatoriamente**:

2.1 Codice Sorgente

- Codice sorgente completo del progetto
- Nessun file mancante
- Nessun codice generato automaticamente escluso senza spiegazione

2.2 Commenti al Codice

Il codice deve essere **adeguatamente commentato**, in particolare nelle parti che implementano:

- Scambio dei messaggi:
 - ELECTION
 - ANSWER
 - COORDINATOR
- Gestione dei timeout (T e T')
- Avvio dell'elezione
- Proclamazione del coordinatore
- Gestione del guasto del nodo P5
- Arresto delle elezioni in corso alla ricezione di COORDINATOR

2.3 Video Dimostrativo

Nel repository GitHub deve essere presente **un video dimostrativo**.

Il video deve mostrare **chiaramente e senza tagli**:

2.3.1 Esecuzione del Sistema

- Tutti e **5 i nodi in esecuzione**
- Ogni nodo in un **terminale separato**
- Log leggibili con l'identificativo del nodo

2.3.2 Prima Elezione

- Avvio dell'elezione
- Elezione corretta di **P5 come coordinatore**
- Ricezione del messaggio COORDINATOR da parte degli altri nodi

2.3.3 Guasto del Coordinatore

- Arresto manuale del processo **P5**
- Evidenza del mancato invio/ricezione di messaggi da P5

2.3.4 Seconda Elezione

- Rilevazione del timeout
- Avvio di una nuova elezione
- Elezione corretta di **P4 come nuovo coordinatore**
- Ricezione del messaggio COORDINATOR da parte degli altri nodi

Suddivisione del Punteggio

Il voto del progetto sarà assegnato come segue:

Componente.....	Peso
Corretto funzionamento del sistema distribuito.....	50%
Video dimostrativo chiaro e corretto.....	30%
Qualità del codice e commenti.....	10%
Documentazione e istruzioni di esecuzione (PDF)....	10%